

Report

Task1,2

先把測資依照格式讀進來，第一行是有幾組，之後每一組開頭的第一行是 node 數，邊數，SD pair 數。整個圖中間連線是無向的，例如 0 4，可以是 0 走到 4 跟 4 走到 0，所以建立雙向圖。之後幾行是 u,v,c 分別是連接的兩個點跟 throughput，而之後幾行是 SD pair，這個是有向的一定要從第一個點傳到第二個點。

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T; //第一行是總測資數
    if(!cin >> T) return 0;
    cout << STUDENT_ID << '\n';
    double sum_throughput = 0.0; //之後用來存總共throughput用來算平均

    for(int inst = 0; inst < T; ++inst){
        int N, M, P; //每組測資第一行是 node數,edge數,SD pair數
        cin >> N >> M >> P;

        vector<tuple<int,int,int>> edges;
        edges.reserve(2 * M); //圖是無向圖要正反邊都建立
        for(int i = 0; i < M; ++i){
            int u, v, c; //起點,終點,throughput
            cin >> u >> v >> c;
            edges.emplace_back(u, v, c);
            edges.emplace_back(v, u, c);
        }

        vector<pair<int,int>> sd(P); //存SD pair
        for(int i = 0; i < P; ++i){
            cin >> sd[i].first >> sd[i].second;
        }

        unique_ptr<MPSolver> solver( //建立 MILP Solver
            MPSolver::CreateSolver("CBC_MIXED_INTEGER_PROGRAMMING"));
        if (!solver) {
            cerr << "Cannot create CBC solver\n";
            return 1;
        }
    }
}
```

建立 $z[u,v]$ 代表的是這條 link 有沒有使用，有的話會設定為 1。

```
// 建立z,之後會設定 z[u,v] = 1 如果link(u->v)有被用
map<pair<int,int>, MPVariable*> z;
for(auto& e : edges){
    int u,v,c; tie(u,v,c) = e;
    z[{u,v}] = solver->MakeBoolVar(
        absl::StrFormat("z_%d_%d", u, v));
}
```

設定 x,y ，分別有三個參數 f,u,v ， f 是哪組 SD pair， $u-v$ 是 link 連線， x 的範圍會被設定在 $0 \sim c$ ，因 $u-v$ link 的最大 throughput 會是 c 。而 y 表示在這組 SD pair 中 $u-v$ 這條 link 有沒有被使用。

```

// 建立x,x[f,u,v]是SD pair 中u-v link的data rate
// 建立y,y[f,u,v]是SD pair 中u-v這條link有沒有被用
map<tuple<int,int,int>, MPVariable*> x, y;
for(int f = 0; f < P; ++f){
    for(auto& e : edges){
        int u,v,c; tie(u,v,c) = e;
        x[{f,u,v}] = solver->MakeNumVar(
            0.0, static_cast<double>(c),
            absl::StrFormat("x_%d_%d_%d", f,u,v));
        y[{f,u,v}] = solver->MakeBoolVar(
            absl::StrFormat("y_%d_%d_%d", f,u,v));
    }
}

```

加入兩條限制， $x \leq c \cdot y$ ，如果要讓 x 有 throughput 的話 y 一定要等於 1 代表有使用這條 link， x 才會最大可以到 c ，之後計算 throughput x 相加才會有值。

$y \leq z$ ，因 z 等於 1 時是代表 $u-v$ link 有被使用，但所有的 SD pair 有其中一個會用，所以 y 可能等於 0 or 1，當 $y=1$ 時，代表 $u-v$ link 有被某組 SD pair 使用，所以 z 一定要等於 1。為 task1 中的 Y-Z binding

```

{ // x-c*y <= 0 得x<=c*y，如果y=0的話代表u-v link沒有被用，所以流量x必=0
  MPCConstraint* ct = //有用的話x的流量最高就是c*1=c
    solver->MakeRowConstraint(-MPSolver::infinity(), 0.0);
  ct->SetCoefficient(x[{f,u,v}], 1.0);
  ct->SetCoefficient(y[{f,u,v}], -static_cast<double>(c));
}
// Y-Z binding
{ // y-z <= 0, 得y<=z, z=1時y這個SD pair不一定有被用，所以可以=0,1
  MPCConstraint* ct = //z=0時，y這個SD pair也一定不能用這條link 所以=0
    solver->MakeRowConstraint(-MPSolver::infinity(), 0.0);
  ct->SetCoefficient(y[{f,u,v}], 1.0);
  ct->SetCoefficient(z[{u,v}], -1.0);
}

```

做流量控管，對每一組 SD pair 分開處理，先看所有非 source 跟 destination 的 node，經過這個 node 的總輸出跟總輸入的量要相同，詳細作法有寫在下面 code 的註解中。

```

for(int f = 0; f < P; ++f){ //對每個SD pair
    int s = sd[f].first, t = sd[f].second;
    for(int u = 0; u < N; ++u){ //對每個node
        if(u == s || u == t) continue; //如果node是source或destination 之後再考慮
        MPConstraint* ct = solver->MakeRowConstraint(0.0, 0.0);
        for(auto& e : edges){ // 對每條邊
            int uu,vv,cc; tie(uu,vv,cc) = e; //把每條邊存到uu,vv,cc
            // x[{f,uu=uu,vv}] 代表從u這個點流出的量(起點是u,終點是vv)，把全部從u這個點流出的量相加
            if (uu == u) ct->SetCoefficient(x[{f,uu,vv}], 1.0);
            // x[{f,uu,vv=u}] 代表從u這個點流入的量(起點是uu,終點是u)，把全部從u這個點流入的量相加
            if (vv == u) ct->SetCoefficient(x[{f,uu,vv}], -1.0);
            // 把全部流出的量減掉流入的量要=0
        }
    }
}

```

處理是 source 跟 destination 的 node，source 要讓全部的輸入流量等於 0，因 source 不會有流量流入，只會有流出。Destination 則相反，要讓流出的流量等於 0，因 destination 不會有流量流出。

```

}
{
    MPConstraint* ct = solver->MakeRowConstraint(0.0, 0.0);
    for(auto& e : edges){
        int uu,vv,cc; tie(uu,vv,cc) = e;
        //找所有邊如果有找到終點是source的,把x即流量全部相加要等於0，因source不會有流量流入
        if (vv == s) ct->SetCoefficient(x[{f,uu,vv}], 1.0);
    }
}
{
    MPConstraint* ct = solver->MakeRowConstraint(0.0, 0.0);
    for(auto& e : edges){
        int uu,vv,cc; tie(uu,vv,cc) = e;
        //找所有邊如果有找到起點是destination的,把x即流量全部相加要等於0，因destination不會有流量流出
        if (uu == t) ct->SetCoefficient(x[{f,uu,vv}], 1.0);
    }
}
}

```

設定 x 範圍要在負無限大到 c 之間，詳細作法在註解中。

```

for(auto& e : edges) {
    int u,v,c; tie(u,v,c) = e;
    MPConstraint* ct = solver->MakeRowConstraint(
        0.0, static_cast<double>(c));
    // 設立範圍是 -無限大到c
    for(int f = 0; f < P; ++f){
        // 對每個SD pair 把u-v這條link的流量全部相加不可以大於c
        // 即u-v link在全部的情形下流量不可以超過c(上限)
        // 但基本上只會有一條路徑通過，所以會把流量全部分給經過的那條
        ct->SetCoefficient(x[{f,u,v}], 1.0);
    }
}
}

```

設定 y 要介於 0-1，對於每一組 SD pair 把 u,v 分開看，考慮 u 時，要把全部從這個 u 點流出的數量要小於等於 1，代表只能有一條流量可以從這個點流出 (u,v)代表 u 是起點，所以看到的是從 u 這個起點流出；考慮 v 時，要把全部從

這個 v 點流入的數量要小於等於 1，代表只能有一條流量可以從這個點流入， (u,v) 代表 v 是終點，所以看到的是流入到 v 這個點，即 task1 中的 single transmitter and receiver per node，因對每個點都最多只會有一個流入與一個流出。

```
// Single transmitter per node, Single receiver per node
for(int f = 0; f < P; ++f){ //對所有SD pair
    for(int u = 0; u < N; ++u){ //對所有node
        MPConstraint* ct = solver->MakeRowConstraint(
            0.0, 1.0);
        //範圍是 負無限大到1
        for(auto& e : edges){
            int uu,vv,cc; tie(uu,vv,cc) = e;
            if(uu == u){ //找所有起點是u的邊
                // y[{f,uu=u,vv}], 全部相加要小於1，即全部從這個點流出的link數量要小於等於1
                ct->SetCoefficient(y[{f,uu,vv}], 1.0);
            }
        }
    }
}

for(int f = 0; f < P; ++f){
    for(int v = 0; v < N; ++v){
        MPConstraint* ct = solver->MakeRowConstraint(
            0.0, 1.0);
        for(auto& e : edges){
            int uu,vv,cc; tie(uu,vv,cc) = e;
            if(vv == v){ //跟上面同樣判斷 y[{f,uu,vv=v}], 全部相加要小於1
                // 從v這個點流入的link數量要小於等於1
                ct->SetCoefficient(y[{f,uu,vv}], 1.0);
            }
        }
    }
}
```

設定 z ，方法與 y 一樣，要讓每一個點的輸出輸入的數量都只有一條，這個部分也是 task1 中的 single transmitter and receiver per node。

```
// 跟上面一樣的方法，對於每個點把流出與流入的link數量要小於等於1，只是改成操作z
// Single transmitter per node, Single receiver per node
for(int u = 0; u < N; ++u){
    MPConstraint* ct = solver->MakeRowConstraint(
        0.0, 1.0);
    for(auto& e : edges){
        int uu,vv,cc; tie(uu,vv,cc) = e;
        if(uu == u){
            ct->SetCoefficient(z[{uu,vv}], 1.0);
        }
    }
}
for(int v = 0; v < N; ++v){
    MPConstraint* ct = solver->MakeRowConstraint(
        0.0, 1.0);
    for(auto& e : edges){
        int uu,vv,cc; tie(uu,vv,cc) = e;
        if(vv == v){
            ct->SetCoefficient(z[{uu,vv}], 1.0);
        }
    }
}
```

計算 throughput，只要讓每組 SD pair 的 source 讓他的全部 $u=s$ 的 x 全部相加最大就好，因中間流量有做平衡與限制，所以最後會自動傳到終點，不用檢查有沒有傳到終點。

```
// 只要看每組的source輸出的總量throughput全部相加讓他最高就好，因有設定中間點是進入流量等於輸出
// 所以進到中間點的流量一定會找到其他點繼續流，直到流到終點
MPObjective* obj = solver->MutableObjective();
for(int f = 0; f < P; ++f){
    int s = sd[f].first;
    for(auto& e : edges){
        int uu,vv,cc; tie(uu,vv,cc) = e;
        if(uu == s){
            obj->SetCoefficient(x[{f,uu,vv}], 1.0);
        }
    }
}
obj->SetMaximization();
```

接著處理輸出，用 sum_throughput 存總共的 throughput，並把所有 z 等於 1 的 link 全部印出來。

```
auto status = solver->Solve();
double total_rate = 0.0; // 把全部的把全部的throughput相加存起來，後面要算平均
if (status == MPSolver::OPTIMAL || status == MPSolver::FEASIBLE) {
    total_rate = solver->Objective().Value();
}
sum_throughput += total_rate;

vector<pair<int,int>> used_links; // 找出有使用的link，看z是不是1
for (auto& pr : z) { // 是的話把該組存出來
    if (pr.second->solution_value() > 0.5) used_links.push_back(pr.first);
}
cout << used_links.size() << '\n'; // 先輸出總共有幾個link被用在依序把有被用的印出
for (auto& uv : used_links) cout << uv.first << ' ' << uv.second << '\n';
```

對每組 SD pair 操作，先算出 throughput 是多少，是 0 的話代表這組 SD pair 沒有被用到，要輸出 0 0。

```
for (int f = 0; f < P; ++f) { // 處理每一組SD pair
    int s = sd[f].first, t = sd[f].second;
    double rate = 0.0;
    for (auto& e : edges) {
        int u,v; double c;
        tie(u,v,c) = e; // 把該組SD pair 的datarate全部相加，如果是0的話就要輸出0 0
        if (u == s) rate += x[f,u,v]->solution_value(); // 代表沒有選用這組
    }
    if (rate < 1e-9) {
        cout << "0 0\n";
        continue;
    }
}
```

建立一個表，把全部相連的邊存進去(用 y 是不是=1 判斷)，之後用 BFS 回溯路徑，並且從 destination 到 source 把所有經過的 node 依序存起來，最後補上 source 這個 node，並把順序相反得到 source 到 destination，並輸出 throughput，使用的 node 數，跟依序走的 node 的路徑，最後再把總 throughput 除上組數印出平均的 throughput。

```
vector<vector<int>> adj(N); // 建立一個表來表示有沒有相鄰
for (auto& e : edges) {
    int u,v; double c;
    tie(u,v,c) = e; // 如果y=1的話代表u-v有使用，存進表中
    if (y[{f,u,v}]->solution_value() > 0.5) adj[u].push_back(v);
}
// 利用BFS回溯路徑，之後要印出每組SD pair經過的路徑
vector<int> parent(N, -1);
queue<int> q;
q.push(s);
parent[s] = s;
while (!q.empty() && parent[t] == -1) {
    int u = q.front(); q.pop();
    for (int v : adj[u]) if (parent[v] == -1) {
        parent[v] = u;
        q.push(v);
    }
}
vector<int> path; // 放destination到source的路徑，最後加入source node在反轉得完整路徑
for (int v = t; v != s; v = parent[v]) path.push_back(v);
path.push_back(s);
reverse(path.begin(), path.end());
// 先印出 throughput 再把用幾個點，整個路徑印出
cout << fixed << setprecision(6) << rate << ' ' << path.size();
for (int node : path) cout << ' ' << node;
cout << '\n';
```

Task3

前面大部分與 task2 一樣方法，先讀取資料，建立雙向圖，把 SD pair 存起來

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T; // 第一行資料是有幾組側資
    if (!(cin >> T)) return 0;
    cout << STUDENT_ID << "\n";
    double sum_throughput = 0.0; // 存總throughput

    for (int inst = 0; inst < T; ++inst) {
        int N, M, P; // 每組側資的第一行是node, edge, SD pair數
        cin >> N >> M >> P;
        // 存u-v link流量
        vector<vector<double>> cap(N, vector<double>(N, 0.0));
        // 用來存這條u-v link還可以用多少flow，一開始是跟最大流量一樣
        vector<vector<double>> rem = cap;
        vector<vector<int>> adj(N); // 用來存u跟v有沒有相鄰
        for (int i = 0; i < M; ++i) {
            int u, v; // 把每條邊的u,v,c讀入，要雙向
            double c;
            cin >> u >> v >> c;
            cap[u][v] = rem[u][v] = c;
            cap[v][u] = rem[v][u] = c;
            adj[u].push_back(v);
            adj[v].push_back(u);
        }
        vector<pair<int, int>> sd(P);
        for (int i = 0; i < P; ++i) { // 存SD pair的起點終點
            cin >> sd[i].first >> sd[i].second;
        }
    }
}
```

Used_tx 與 used_rx 用來存有沒有當過傳送端與接收端，用來實作 Single transmitter per node 與 Single receiver per node 一樣的限制

```
// used_tx與used_rx用來存有沒有當過傳送端與接收端
// 跟ortools的single transmitter, receiver一樣
vector<bool> used_tx(N, false), used_rx(N, false);
// 存每組SD pair的最終流量
vector<double> rates(P, 0.0);
vector<vector<int>> paths(P);
```

處理每組 SD pair，best 用來存從 s 開始到該點的最佳 throughput，parent 是用來回溯路徑，建立一個 queue，存(throughput,node)，檢查目前的點是不是到終點是，是的話不用繼續做直接停止，不是則先拿出 u-v 這個 link 還有多少 throughput 可以用，如果小於 0 或 u 當過起點 v 當過終點，也要跳過因不合，接著拿還可以用的量跟原本前一個 node 的最佳量的最小值，因 throughput 要取最小的，去更新 bset[v]，把所有可能都跑過一輪會更新出最佳的情形。有被選用到的話會把這個點 push 進去 queue 中，之後繼續處理與他相鄰的 node。

```
for (int f = 0; f < P; ++f) {
    int s = sd[f].first, t = sd[f].second;
    // best用來存從s開始到該點的路徑中最佳的throughput
    vector<double> best(N, 0.0);
    vector<int> parent(N, -1);
    // 起點預設為無限大
    best[s] = numeric_limits<double>::infinity();
    priority_queue<pair<double, int>> pq;
    pq.push({ best[s], s }); //先把(無限大,s)存進去

    while (!pq.empty()) {
        auto [b, u] = pq.top(); pq.pop();
        if (b < best[u]) continue; // 如果不一樣的話需跳過
        if (u == t) break; // 如果s=t代表到終點了，不用繼續看
        for (int v : adj[u]) { // 找所有與現在拿出的u點相鄰的點
            // 拿出從現在的點到他的鄰居點剩餘多少flow可以用
            double rcap = rem[u][v];
            // 如果剩餘flow已經小於0，或u當過傳送端，v當過接收端，需跳過
            // 因同一個點不可以當兩次接收或傳輸端
            if (rcap <= 0 || used_tx[u] || used_rx[v]) continue;
            double nb = min(b, rcap); // 拿出較小的代表是可以用的flow量
            if (nb > best[v]) { // 有比走到的點原本的好的話做更新
                best[v] = nb; //一開始best[v]會是0，所以會被更新成最大可以走的情形
                parent[v] = u;
                pq.push({ nb, v });
            }
        }
    }
}
```

Rate 是到終點時的 throughput，即這條 link 的 throughput 量，如果大於 0 代表這組 SD pair 有被選用，用 parent 回溯路徑並存起來，把現在已經使用的量扣掉得到剩餘可使用的量，並更新路徑中的各點已經當過起點或終點。

```
double rate = best[t]; // 到終點可以用的throughput
if (rate > 0) {
    vector<int> path;
    for (int v = t; v != s; v = parent[v])
        path.push_back(v);
    path.push_back(s); // 建立路徑，先一路回溯最後加上source點 再反轉
    reverse(path.begin(), path.end());
    rates[f] = rate; // 存throughput
    paths[f] = path; // 存路徑
    for (int i = 0; i + 1 < (int)path.size(); ++i) {
        int u = path[i], v = path[i + 1];
        rem[u][v] -= rate; // 算完一組SD pair，要更新數值
        used_tx[u] = true; // 更新剩下的流量為扣掉現在已經用的
        used_rx[v] = true; // 更新u點已經當過傳送點，v點已經當過接收點
    }
}
```

只要剩餘的量小於原本的量，代表這條 link 有被使用到，先印出總共用了多少 link 數，之後印出前面判斷出有使用的 link。

```
// 存所有真的有使用的link，後續要印出
vector<pair<int, int>> used_links;
for (int u = 0; u < N; ++u) {
    for (int v : adj[u]) {
        if (rem[u][v] < cap[u][v]) {
            used_links.emplace_back(u, v);
        }
    }
}
// 每筆側資要先輸出link使用的數量，再輸出u-v link
cout << used_links.size() << "\n";
for (auto& e : used_links) {
    cout << e.first << " " << e.second << "\n";
}
```

計算總 throughput 並在最後平均輸出，我把輸出固定在小數點後六位，並輸出每組 SD pair 運算出的結果，throughput=0 代表沒有取用這組，要輸出 0 0，throughput > 0，則要輸出 throughput 量，用了幾個 node，路徑怎麼走。

```
// 計算總throughput
double total_rate = 0.0;
for (double r : rates) total_rate += r;
// 輸出每組SD pair的結果
for (int f = 0; f < P; ++f) {
    if (rates[f] <= 0) { // 如果沒有被採用，輸出0 0
        cout << "0 0\n";
    }
    else {
        auto& path = paths[f];
        //有被用的話，輸出throughput，用了幾個node跟路徑
        cout << static_cast<int>(rates[f]) << " " << path.size();
        for (int x : path) cout << " " << x;
        cout << "\n";
    }
}
// 輸出該組側資的throughput(我限制為整數)，如果不是最後一筆側資都要換行
cout << static_cast<int>(total_rate) << (inst < T - 1 ? "\n" : "");
sum_throughput += total_rate;
}

cout << "\n" << fixed << setprecision(6) << (sum_throughput / T) << "\n";
```

Question

1.

Y-Z binding 我用 $y \leq z$ 做限制，當 link 有被指派到 SD pair 的話，此時 $y=1$ ，由前面的限制，會自動把 z 也要等於 1。Single transmitter per node 與 Single receiver per node 我把所有的 y, z 都拿出來， y 分 SD pair 去考慮所有的 u 時，輸出的數量要小於等於 1，考慮 v 時輸入的數量要小於等於 1； z 則是全部的 $u-v$ link，所以一樣分 u, v 考慮，當考慮 u 時，輸出的數量要小於等於 1，考慮 v 時輸入的數量要小於等於 1。

2.

用 ottools 算出的 average throughput 是 111.2

用 myalgo 算出的 average throughput 是 98.2

Ratio = $98.2 / 111.2 = 0.883093$

3.

我使用與 task1,2 類似的限制，要讓每個點都只能當一次進出點。整體邏輯是直接從第一組 SD pair 開始判斷能不能走，如果可以的話就直接走這條，其中利用持續更新鄰居的方法可以讓這條 link 有最好的 throughput，之後就照 SD pair 的順序判斷，如果有可以走的話，會直接取這組 SD pair 來走，直到考慮完全部的 SD pair。我沒有先全部算完之後從最大 throughput 的 SD pair 開始走等方法，是直接照順序考慮 SD pair 並讓現在考慮的 SD pair 可以有最大的 throughput，雖然有許多漏洞，但有比 baseline 好。

4.

前面建立存流量與剩下流量需要建立 $O(N^2)$ 大小，中間主要計算的部分，考慮每一組 SD pair，其中幾乎每個點都會被我考慮、更新一遍，之後會找出有幾個 node 有在路徑中，並做更新路徑與 node 假設當作 $\log(N)$ ，所以這段計算花了 $O(P * (N + \log(N)))$ 的複雜度，後續輸出的部分，最多也只可能是 $O(P * N)$ 用來輸出每組 SD pair 的邊，最差有可能全部的點是一條路徑，所以路徑最長是 N ，所以整體而言，最大的為 $O(P * (N + \log(N)))$

Initial commit 我只更新 student id

Second commit 我把 task1,2 的部分全部完成

Final commit 我有把前面 task1,2 的輸出方法換掉，我一開始只有輸出 throughput 沒有輸出其他東西，後面更正，並完成 task3 與 report 的部分

最後附上 accept 的圖片

111550136 submissions							
#	When	Who	Problem	Lang	Verdict	Time	Memory
319913584	May/16/2025 23:28 ^{UTC+8}	111550136	A - netcap-lab5	C++17 (GCC 7-32)	Accepted	77 ms	100 KB
319913249	May/16/2025 23:25 ^{UTC+8}	111550136	A - netcap-lab5	C++17 (GCC 7-32)	Wrong answer on test 3	77 ms	0 KB
319913130	May/16/2025 23:24 ^{UTC+8}	111550136	A - netcap-lab5	C++17 (GCC 7-32)	Wrong answer on test 2	31 ms	0 KB
319911786	May/16/2025 23:14 ^{UTC+8}	111550136	A - netcap-lab5	C++17 (GCC 7-32)	Wrong answer on test 1	46 ms	0 KB
319910630	May/16/2025 23:05 ^{UTC+8}	111550136	A - netcap-lab5	C++17 (GCC 7-32)	Wrong answer on test 3	61 ms	0 KB